Rapid Generalization of Motion Planning for Robot Manipulator Among Locally Changed Environments

Xiangjian Li, Xinjie Xiao, Huashan Liu, *Senior Member, IEEE*, and Bo Shen, *Senior Member, IEEE*

***Abstract*—As is known, the problem of making a trained deep reinforcement learning (DRL) model to adapt to a new environment different from where it trains remains open. In this article, DRL-based techniques are investigated and deployed for the robotic motion planning task in locally changed environments. First, a dynamic-entropy actor-critic is proposed to automatically adjust the coefficient of the entropy cluster to enable the agent to efficiently learn the potentially optimal policy. Second, a tutoring-guiding mechanism is established to heuristically explore and sufficiently exploit valuable experience. Third, a mechanism consisting of progressive planner and inverse kinematics mapper is designed to cope with localized changes in the environment, which enables the robot manipulator to rapidly adapt to the newly changed environment. Finally, experimental results have verified the superior performance of the proposed approaches.**

***Index Terms*—Deep reinforcement learning (DRL), environment, localized change, motion planning, robot manipulator.**

I. Introduction

A. Motivation

DEEP reinforcement learning (DRL) has been employed broadly in many fields [1], [2], as one of the most essential fields, motion planning of robots based on DRL appeals to increasing researches [3], [4].

In complicated environments, it is challenging to plan moving trajectory for robot with redundant degrees of freedom (DoFs) to adapt to different tasks with satisfactory performance. Moreover, it has been verified that obtaining a sufficient amount of accurate pretraining data or valuable human experience and knowledge for robots can speed up the training [5], [6], but such process is time-consuming and expensive, and motion planning tasks with multiobjective optimization exacerbate this situation. In addition, even small localized changes in the environment are likely to invalidate the results of previously trained motion planning policy. Although retraining or replanning can tackle such issues, they are inefficient and require extra pretraining data to accelerate the training. In various tasks, the robot (agent) equipped with DRL is expected to adapt to different environments and obtain the maximum episode return. Although the exploration capability of DRL under the maximum entropy framework can be improved to be competent in different environments by adjusting the entropy of the policy, it is expensive to manually pick off the most feasible entropy through extensive trials.

Since the exploration of DRL is guided by the reward, the exploitation of the experience in terms of augmentation and sampling of experience and the corresponding reward mechanism can heavily affect the policy learning [7], [8], especially in situations where the environment changes after the training has been completed, the problem on how to correct the trained results so as to rapidly adapt to the new environment remains open.

B. Related Works

Under the maximum entropy framework, both the reward and the entropy of the policy are taken into account in the episode return, which can facilitate the exploration [9]. Moreover, for different tasks, proper entropies can boost the efficiency of the policy learning [10]. Nonetheless, manually adjusting the entropy is imprecise and inefficient, and the potential optima are likely to be neglected. Although additional structure or functions can be imposed into the updates to facilitate DRL in learning the optimal policy [11], [12], a fixed entropy fails to strongly incentivize the agent on its exploratory actions [13]. The entropy is expected to be dynamically adjusted according to the policy learning, since different entropies have different effects for the agent to explore the environment. Moreover, excessive useless exploratory behaviors should be constrained to drive the agent to learn the potentially optimal policy and accomplish the task.

In the policy learning, although high-quality curricula or demonstrations can assist the agent to converge rapidly [14], [15], it is difficult to get plenty of precise experience that can

Received 23 October 2025; revised 17 November 2025; accepted 21 November 2025. Date of publication 24 December 2025; date of current version 5 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62373095 and Grant 62273088, in part by the Shanghai Oriental Talent Program (Outstanding Project), in part by the Discipline Innovation Cultivation Program under Grant XKCX202304, and in part by the Fundamental Research Funds for the Central Universities under Grant 2232022G-09. Paper no. TII-25-7494. *(Corresponding author: Huashan Liu.)*

The authors are with the School of Information and Intelligent Science, Donghua University, Shanghai 201620, China, and also with the Engineering Research Center of Digitized Textile and Apparel Technology, Ministry of Education, Donghua University, Shanghai 201620, China (e-mail: xjli@mail.dhu.edu.cn; xinjie.xiao@mail.dhu.edu.cn; hsliu@dhu.edu.cn; bo.shen@dhu.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TII.2025.3637078, provided by the authors.

Digital Object Identifier 10.1109/TII.2025.3637078

1941-0050 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 02,2026 at 09:12:06 UTC from IEEE Xplore. Restrictions apply.

act as demonstrations. The problem of methods for obtaining heuristic experience and the balance between exploration and exploitation remains open. Moreover, due to the randomness in the sampling update, high-quality experience in the policy learning might be neglected, which results in the agent for repeated explorations and updates, or even being stranded in suboptimal policies. Since DRL is reward-guided, the reward distribution can significantly impact the direction of the policy learning. In addition, the reward distributions in different environments are difficult to design manually, and a static reward may result in the previous valuable experience being neglected in the stochastic sampling updates. Due to this, the agent can be encouraged to learn the optimal policy by projecting the valuable experience into the reward distribution.

As known, it is difficult for a trained policy in a given scenario to adapt to a different scenario, even if the scenario has localized changes. Transfer learning is always recalled at this point, to be used to solve the problem of policy inapplicability due to differences between scenarios [16], [17]. However, the situation is not that simple in fact and the intricacies of knowledge transfer are notably amplified in fields of DRL [18], [19]. In particular, it seems too costly when transfer learning is used, to generalize situations applicable when there are small changes in the environment. Thus, it is necessary to find a more targeted and simpler approach than transfer learning . Approaches based on artificial neural networks (ANNs) are kinds of imitation or fitting means in continuous spatiotemporal space of the robot [20], [21], [22]. These kinds of methods of ANNs generally based on traditional kinematics formulas or pretrained model, which makes them restricted by the geometry of robots and expensive pretraining data. In addition, for localized changes in the environment, retraining and replanning result in a serious waste of resources and the benefits may be undesirable, therefore, a rapid and simple method to mend the previous trajectory that can satisfy multiple optimization objectives is worth investigating.

C. Contributions and Organization of this Article

In response to the aforementioned discussions, we aim to provide general motion planning for robot manipulator, from the perspective of enhancing DRL for the policy learning and coping with the adaptation of trained results in a newly changed environment. The main contributions are summarized as follows.

1) A dynamic-entropy actor-critic (DEAC) with sharpness-aware minimization (SAM) is proposed to assist the agent in adapting to different environments and to boost the efficiency of the policy learning. The entropy cluster is automatically adjusted by analyzing the policy learning in the explored region, as it can be reflected by the temporal difference (TD) error, to encourage the agent to make explorations. Moreover, the TD error threshold is automatically adjusted to protect the agent from learning suboptimal policies due to the overweighted entropy in the episode return.
2) A tutoring-guiding (TG) mechanism is proposed to accelerate the convergence of the policy learning and assist the agent to sufficiently exploit the previous valuable experience. The tutoring in the TG is built into DRL to generate heuristic experience for shortening paths based on explored experience in the early stage of the training. The guiding in the TG can adjust the reward distribution according to the previous valuable experience to prevent the potentially optimal policy from being neglected and drive the agent to approach the optimal policy.
3) A rapid refitting program consisting of a progressive planner (PP) and an inverse kinematics mapper (IKM) is proposed, to cope with the localized changes in the environment without retraining DRL. The PP is based on Voronoi diagram and Dijkstra planner, and is used to seek for alternative paths to adapt to the locally changed environments. The IKM with a mixed normal distribution algorithm is used to map the path in the Cartesian space to the trajectory in the joint space of the robot while ensuring that the inverse kinematics (IK) solution is energy-optimal.

The rest of this article is organized as follows. Section II describes the fundamentals of the proposed algorithm DEAC. In Section III, the details of TG for the efficient policy learning is elaborated. In Section IV, a rapid refitting program consisting of PP and IKM for locally changed environments is formulated. Experimental results are given in Section V. Finally, Section VI concludes this article.

II. Dynamic-Entropy Actor-Critic (DEAC)

The process of DRL-based motion planning of a robot can be described as a Markov decision process, and it can be represented as a tuple $\boldsymbol{\mathcal{M}} = (\boldsymbol{\mathcal{S}}, \boldsymbol{\mathcal{A}}, F, \mathrm{T}, r, \tau)$, where $\boldsymbol{\mathcal{S}}$ is the state space and $\boldsymbol{\mathcal{A}}$ is the action space, both of them are assumed to be continuous. $F(\boldsymbol{s}_{t+1}|\boldsymbol{s}_t, \boldsymbol{a})$ is the transition probability of the environment. $t \in \{1, 2, \ldots, \mathrm{T}\}$ denotes the number of the transition times and T is the total number of that. In an interaction between the agent and the environment, the state $\boldsymbol{s}_t \in \boldsymbol{\mathcal{S}}$ will be transited to $\boldsymbol{s}_{t+1} \in \boldsymbol{\mathcal{S}}$ when the agent takes action $\boldsymbol{a}_t \in \boldsymbol{\mathcal{A}}$ according to the policy π. r is the reward that will emit a single-step reward r_t when the agent takes an action at the tth interaction. $\tau \in (0, 1)$ is a discount that determines the priority of short-term rewards.

Under the maximum entropy framework, the primary formulas of DRL can be expressed as

$$J_{\boldsymbol{\eta}} = \mathbb{E}_{\boldsymbol{a}_t, \boldsymbol{s}_t, \boldsymbol{s}_{t+1} \sim \boldsymbol{\mathcal{U}}} \left\{ \frac{1}{2} \left[Q(\boldsymbol{a}_t, \boldsymbol{s}_t) - r_{t+1} - \tau \left(\min_{i=1,2} Q'_i(\boldsymbol{a}_{t+1}, \boldsymbol{s}_{t+1}) + \alpha S_\lambda(\boldsymbol{s}_{t+1}) \right) \right]^2 \right\} \quad (1)$$

$$J_{\boldsymbol{\phi}} = \mathbb{E}_{\boldsymbol{s}_t, \boldsymbol{a}_t \sim \boldsymbol{\mathcal{U}}} \left[- \min_{i=1,2} Q_i(\boldsymbol{s}_t, \boldsymbol{a}_t) - \alpha S_\lambda(\boldsymbol{s}_t) \right] \quad (2)$$

where $J_{\boldsymbol{\eta}}$ and $J_{\boldsymbol{\phi}}$ are principles for updating the critic and the actor networks, respectively. $\boldsymbol{\eta}$ and $\boldsymbol{\phi}$ are the entire parameters of the corresponding networks. $\mathcal{U}$ is the memory for storing experience during the interactions. Q' denotes the output of the target critic network. α is the temperature parameter. $S_\lambda(\cdot)$ denotes a monotonically increasing entropy belonging to the entropy cluster $S(\cdot)$ with coefficient λ.

Since SAM [23] can seek the point of maximum loss around the current parameters of the ANNs, it is used in the updates

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 02,2026 at 09:12:06 UTC from IEEE Xplore. Restrictions apply.

of DRL to enhance stability of the ANNs. As illustrated by Tsallis actor-critic with clipped automatic entropy adjustment (TAC-CA) [10], desirable entropies do much favor to enhance the DRL performance in diverse tasks. However, attempting entropies individually is time-consuming and expensive. Moreover, exploring the environment and exploiting the experience have much randomness, which are challenges for manually selecting the desirable entropy.

To deal with the aforementioned dilemma, a DEAC is proposed in this article to assist the agent in automatically adjusting the coefficient of the entropy cluster in the training, the principle for adjusting the coefficient is defined as

$$\lambda \leftarrow \lambda + \mathrm{C}_\lambda \left[1 - \tanh(\upsilon / H_\upsilon)\right], \lambda \in [\lambda_{\min}, \lambda_{\max}] \quad (3)$$

where $\upsilon = \frac{1}{2}[Q(\boldsymbol{a}_t, \boldsymbol{s}_t) - r_{t+1} - \tau(\min_{i=1,2} Q_i'(\boldsymbol{a}_{t+1}, \boldsymbol{s}_{t+1}) + \alpha S(\boldsymbol{s}_{t+1}))]^2$ is the TD error. H_υ is the TD error threshold and λ will be adjusted when $\upsilon < H_\upsilon$. $\lambda_{\min}$ and $\lambda_{\max}$ are the minimum and maximum values for λ, respectively. C_λ is a constant.

As defined in (1), the critic networks update to reduce the TD error, which enables DRL to obtain the maximum episode return so as to learn the optimal policy. As the TD error reduces, DRL can be considered to gradually learn the policy in the explored region. In such situation, λ can be increased for enhancing the agent's exploratory behaviors for seeking the potentially optimal policy in extra region.

Furthermore, under the maximum entropy framework, the episode return is extended as the sum of the reward and the entropy, which can be described as $\sum_{t=1}^{\mathrm{T}} \mathbb{E}_{\boldsymbol{s}_t, \boldsymbol{a}_t \sim \boldsymbol{\mathcal{U}}}[r(\boldsymbol{s}_t, \boldsymbol{a}_t) + \alpha S_\lambda(\boldsymbol{s}_t)]$, and the policy learning is guided by the reward and the entropy. Importantly, even though the entropy can facilitate the agent's exploratory behaviors, the aim of the agent is to accomplish the task; thus; in an episode return, the reward should play a more dominant role than the entropy for the vast majority of training time.

The restriction should be imposed such that the increase in the entropy is controllable, and it is defined as

$$H_\upsilon \leftarrow H_\upsilon - \mathrm{C}_{H_\upsilon} \left[1 - \tanh(\upsilon / H_\upsilon)\right], H_\upsilon \in [H_{\min}^\upsilon, H_{\max}^\upsilon] \quad (4)$$

where C_{H_υ} is a constant, and $H_{\min}^\upsilon$ and $H_{\max}^\upsilon$ are the minimum and maximum values of H_υ, respectively.

In the training, as shown in (3), the entropy coefficient λ is adjusted according to the TD error. When the TD error is smaller than the TD error threshold H_υ, the policy in the explored region is considered to be well learned by the agent, in which case λ will be increased to encourage the agent to explore extra region. As expressed by (4), a gradually tightened H_υ can constrain the proportion of the entropy in the episode return, and motivate the agent to sufficiently explore the potential optimal policy by using the enhanced exploratory behaviors. Moreover, the agent will be activated again for exploration until the current H_υ is satisfied. Importantly, according to (3) and (4), the principles for automatically adjusting the entropy ensure the balance of exploration and exploitation in the policy learning of DEAC. The proposed DEAC is presented in Algorithm 1.

Remark 1: The reward obtained by the agent at a given moment depends on the learned policy, while different entropies have different effects on the episode return and the direction of the policy learning. In detail, an excessively large entropy will dominate the episode return and misguide the agent to concentrate on learning to increase the entropy, which will cause the task to fail. In the meantime, a small entropy is difficult to encourage the agent to make more exploratory behaviors to seek the optimal policy. Thus, it is crucial to dynamically regulate the growth of entropy.

Algorithm 1: DEAC.

```
Initialize the actor and critic networks, the memory, temperature parameter α, entropy coefficient λ, TD error threshold H_υ, G episodes and T steps;
for g = 1, 2, ··· , G do
    Initialize state s_t, where t = 0;
    for t = 1, 2, ··· , T do
        Get action a_t by the actor network;
        The agent interacts with the environment by taking action a_t and gets the next state s_{t+1} and reward r_{t+1};
        Store s_t, a_t, s_{t+1} and r_{t+1} in the memory;
        Replace s_t with s_{t+1};
        Update the critic and actor networks with intervals by (1) and (2), respectively;
    Calculate TD error υ;
    if υ < H_υ then
        Update λ according to (3);
        Update H_υ according to (4);
        Update entropy cluster S(·).
```

Remark 2: With increasing λ, the agent's exploratory behaviors are enhanced and the entropy becomes larger. To avoid the policy learning being misguided, the TD error threshold is automatically adjusted to force the agent to obtain the maximum episode reward and accomplish the task. Moreover, since the region explored by the agent is enlarged, a stricter constraint in terms of a smaller TD error threshold can facilitate the sufficiency of the policy learning.

III. TG Mechanism

Heuristic is efficient in enhancing policy learning, thus the way in which heuristic experience is generated and utilized is particularly important. Moreover, when the agent learns a superior policy to the previous ones, the commonly used static reward distribution may result in the agent not being able to take sufficient advantage of the newly gained policy.

To cope with these issues, a TG mechanism is proposed in this article to simplify the explored transitions and adjust the reward distribution, so as to motivate sufficient utilization of previous experience, and finally, boost the efficiency of the policy learning.

A. Tutoring

In the tutoring, as shown in Fig. 1, a long-term memory is used to store long-term experience in the training, and a tutoring memory is used to store the heuristic experience. In training, when the robot discovers feasible trajectories that can connect the initial state to the target state without collision, these

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 02,2026 at 09:12:06 UTC from IEEE Xplore. Restrictions apply.

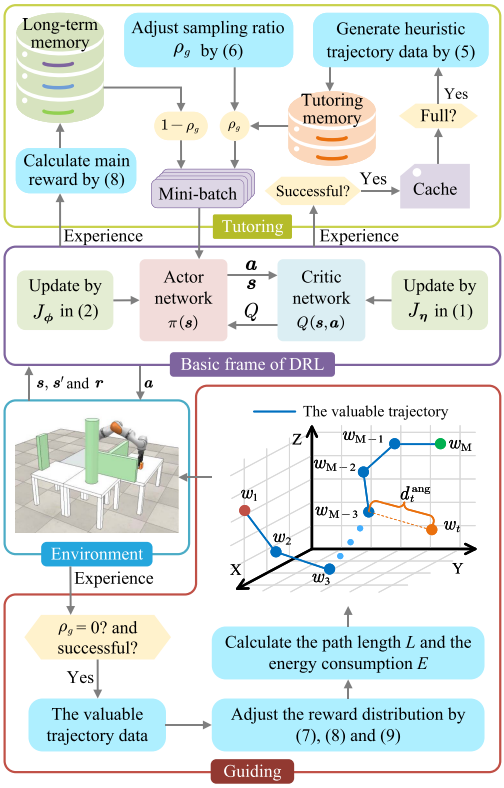


Fig. 1. Schematic diagram of the proposed TG. The proposed TG mechanism is divided into the two components of *Tutoring* and *Guiding*. In *Tutoring*, expert data are stored in the tutoring memory, while explored data are stored in the long-term memory. During updates, data in the minibatch will be sampled from these two memories adjusted by ρ_g. As ρ_g decreases, the proportion of data from the long-term memory gradually increases. If $\rho_g = 0$ and a successful trajectory is found, *Guiding* will be activated. According to the explored valuable trajectories, the reward is continuously adjusted through *Guiding*, promoting the agent to leverage previous valuable experience.

trajectories are regarded as successful trajectories. In addition, the cache is used to store the successful trajectory experience temporarily for the tutoring memory.

In the early stage of the training, the tutoring will be activated when the cache is fully filled with the experience of successful trajectories. In this stage, the state transitions of the trajectory are inefficient since the learned policy is poor. To tackle this issue, in the tutoring, the transitions are progressively simplified by the principle described as

$$\boldsymbol{s}_{t+n} \xleftarrow{\boldsymbol{a}_{t,t+n}} \boldsymbol{s}_t,\ \exists \boldsymbol{a}_{t,t+n} \in \boldsymbol{\mathcal{A}},\ |\boldsymbol{a}_{t,t+n}| \le |\boldsymbol{a}_{\max}| \tag{5}$$

where $n \in \{2, \ldots, \mathrm{N}\}$, and N is the maximum attempting interval, $(t+\mathrm{N}) \le \mathrm{T}$. $\boldsymbol{a}_{\max}$ is the maximum value of the action that can be taken by the agent.

In (5), during trajectory simplification, the robot attempts to reach the $(t+n)$th state directly from the tth state. It is a loop, where n varies from 2 to N. Moreover, within this loop, when the robot cannot reach the $(t+n)$th state from the tth state, the loop will be terminated. The connection from the tth state to the $(t+n-1)$th state will be created and the corresponding experience data will be saved. The next loop begins at the $(t+n-1)$th state. Furthermore, when the robot attempts to reach the $(t+n)$th state directly from the tth state, each joint angle $\boldsymbol{a}_{t,t+n}$ should be less than the maximum angle $\boldsymbol{a}_{\max}$, i.e., $|\boldsymbol{a}_{t,t+n}| \le |\boldsymbol{a}_{\max}|$, which can bridge states $\boldsymbol{s}_t$ and $\boldsymbol{s}_{t+n}$. This approach not only satisfies the joint angle constraints for the robot at each step, but also meets the settings for the range of actions in DRL. Such experience data can be effectively utilized as heuristic experience data for the policy learning.

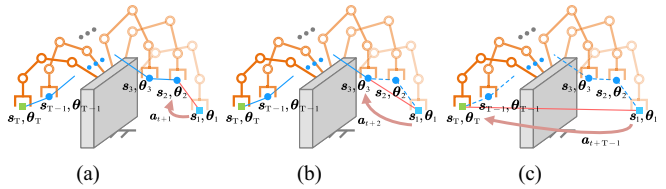


Fig. 2. Diagram of the trajectory simplification. In (a), when $n = 1$, the simplified path is identical to the original one. In (b), when $n = 2$, the trajectory is attempted to be simplified by skipping $\boldsymbol{s}_2$. In (c), when $n = \mathrm{T}-1$, the trajectory is attempted to connect the starting state $\boldsymbol{s}_1$ directly to the ending state $\boldsymbol{s}_\mathrm{T}$, and all other nodes are skipped.

In Fig. 2, specific cases with different values of n are shown. During trajectory simplification, when $n = 1$, as depicted in Fig. 2(a), state $\boldsymbol{s}_1$ is only attempted to be directly connected to the next state $\boldsymbol{s}_2$. This results in the simplified trajectory remaining unchanged. In Fig. 2(b), state $\boldsymbol{s}_1$ is attempted to be directly connected to state $\boldsymbol{s}_3$, with an interval of one state between them. In this case, if an action $|\boldsymbol{a}_{1,3}| = |\boldsymbol{\theta}_3 - \boldsymbol{\theta}_1|$ exists, then state $\boldsymbol{s}_2$ will be skipped to simplify the trajectory. Furthermore, in the extreme case where $n = \mathrm{N} = \mathrm{T}-1$, as shown in Fig. 2(c), state $\boldsymbol{s}_1$ is attempted to be directly connected to state $\boldsymbol{s}_\mathrm{T}$. However, in this case, the robot could collide with obstacles and no action $|\boldsymbol{a}_{1,\mathrm{T}}| = |\boldsymbol{\theta}_\mathrm{T} - \boldsymbol{\theta}_1| \le |\boldsymbol{a}_{\max}|$ exists. Hence, the range of n is set to start from 2, and N should not be set excessively large to satisfy $|\boldsymbol{a}_{t,t+n}| \le |\boldsymbol{a}_{\max}|$.

As shown in Fig. 1, the experience in the minibatch is sampled from the tutoring memory and the long-term memory, and the sampling ratio is automatically adjusted as

$$\rho_g = \begin{cases} \rho_g + \tau, & \mathrm{G}_0^\rho \le g \le \mathrm{G}_{\max}^\rho/2 \\ \rho_{\max} - \tau, & \mathrm{G}_{\max}^\rho/2 < g < \mathrm{G}_{\max}^\rho \end{cases} \tag{6}$$

where $g, \mathrm{G}_0^\rho, \mathrm{G}_{\max}^\rho \in \{1, 2, \ldots, \mathrm{G}\}$ and G is the maximum number of episode. ρ_g is the sampling ratio of experience from the tutoring memory in each episode, while the sampling ratio for the experience sampled from the long-term memory is $(1-\rho_g)$. τ is a constant. G_0^ρ is the number of episode when the tutoring is activated. $\rho_{\max}$ is the maximum sampling ratio and $\mathrm{G}_{\max}^\rho$ is the maximum number of episodes to use the tutoring.

In early stage of the training, when the agent learns a trajectory to reach the objective, the state transitions in this trajectory will be simplified through (5) as the heuristic experience that is stored in the tutoring memory. In this stage, heuristic experience is dominant in the minibatch and it can motivate the agent to explore potentially shorter trajectories. In the later stage of the training, the agent has learned the capability of simplifying trajectories from the heuristic experience. To allow the agent to exploit the capability and explore independently, as defined in (6), the ratio of the heuristic experience sampled from the tutoring memory gradually decreases, which makes the agent gradually disengage from the effect of the heuristic experience. Correspondingly, the ratio of the experience sampled from the long-term memory gradually increases. Thereout, the policy

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 02,2026 at 09:12:06 UTC from IEEE Xplore. Restrictions apply.

learned in the later stage of the training will be ensured to be unbiased.

Remark 3: Heuristic experience is typically biased, thus the tutoring memory is only used to store the heuristic experience in the early stage of the training, as defined in (6), in the updates, the sampling ratio of the heuristic experience is gradually reduced to 0, which ensures that the policy learning is shifted from biased to unbiased.

B. Guiding

Since the policy learning is reward-guided and the experience sampling for updates is stochastic, potential valuable experience could be neglected during the training. To alleviate this issue, as shown in Fig. 1, when the agent obtains a feasible trajectory, which is considered as the valuable trajectory, it will be sufficiently exploited in the subsequent exploration by adjusting the reward distribution.

The valuable trajectory can be discretized as a set of nodes; for each node, the information it contains can be described as

$$\boldsymbol{w}_m = \{\boldsymbol{p}_m, \boldsymbol{o}_m, \boldsymbol{\theta}_m\} \tag{7}$$

where $m \in \{1, 2, \ldots, \mathrm{M}\}$ is the number of the node in the valuable trajectory and M is the maximum number, and $\boldsymbol{p}_m = \{x_m, y_m, z_m\}$ and $\boldsymbol{o}_m = \{\alpha_m, \beta_m, \gamma_m\}$ denote the position and the orientation of the node $\boldsymbol{w}_m$, respectively. x_m, y_m, and z_m are the coordinates in the Cartesian space, and α_m, β_m, and γ_m are Euler angles. $\boldsymbol{\theta}_m$ is the set of the joint angles.

In the training, the path length L and the energy consumption E of the trajectory are calculated as

$$L = \sum_{m=1}^{\mathrm{M}-1} \left\| \boldsymbol{p}_{m+1} - \boldsymbol{p}_m \right\|_2 \tag{8}$$

$$E = \sum_{m=1}^{\mathrm{M}-1} \left\| \boldsymbol{\omega} \odot |\boldsymbol{\theta}_{m+1} - \boldsymbol{\theta}_m| \right\|_1 \tag{9}$$

where $\boldsymbol{\omega}$ is the vector of weights for the joints in terms of inertia with its elements are assigned according to the principle that the greater the moment of inertia of a specific joint, the bigger the value of corresponding element of $\boldsymbol{\omega}$.

The TG reward is designed to adjust the reward distribution, which can be defined as

$$r(t) = f(t)\left[1 + \frac{f(t)}{|f(t)|} g(t)\right] \tag{10}$$

$$f(t) = -\mathbf{A}[\boldsymbol{e}_t^{\mathrm{pos}} \cup \boldsymbol{e}_t^{\mathrm{ori}}]^{\mathrm{T}} - C^{\mathrm{obs}}\left(1 - \frac{d_t^{\mathrm{obs}}}{d_{\max}^{\mathrm{obs}}}\right) + C^{\mathrm{col}} \tag{11}$$

$$g(t) = \frac{1}{2} C^{\mathrm{flag}} \left[\mathrm{clip}\left(1 - \frac{L}{\mathrm{L}_{\max}}\right) + \mathrm{clip}\left(1 - \frac{E}{\mathrm{E}_{\max}}\right) \right] \exp\left(-d_t^{\mathrm{ang}}\right) \tag{12}$$

where $f(t)$ is the main reward, and $g(t)$ is the additional reward used to adjust the reward distribution. $\mathbf{A}$ is a constant matrix. $\boldsymbol{e}_t^{\mathrm{pos}} = |\boldsymbol{p}_t - \boldsymbol{p}^{\mathrm{obj}}|$ and $\boldsymbol{e}_t^{\mathrm{ori}} = |\boldsymbol{o}_t - \boldsymbol{o}^{\mathrm{obj}}|$ denote the position and orientation errors, respectively. $\boldsymbol{p}_t$ and $\boldsymbol{o}_t$ are the tth position and orientation of the robot end, respectively. $\boldsymbol{p}^{\mathrm{obj}}$ and $\boldsymbol{o}^{\mathrm{obj}}$ are the objective position and orientation, respectively. d_t^{obs} is the minimum distance between the robot plant and the obstacles, and $d_{\max}^{\mathrm{obs}}$ is the maximum value of the distance under consideration. L and E are the path length and the energy consumption of the current valuable trajectory, respectively, and $\mathrm{L}_{\max}$ and $\mathrm{E}_{\max}$ are the corresponding maximum values of them. $d_t^{\mathrm{ang}} = \min\|\boldsymbol{\theta}_t - \boldsymbol{\theta}^{\mathrm{val}}\|$ denotes the minimum distance between the tth angle set $\boldsymbol{\theta}_t$ and the angle set $\boldsymbol{\theta}^{\mathrm{val}}$ in the valuable trajectory. C^{obs}, C^{col}, and C^{flag} are sign functions, i.e., $C^{\mathrm{obs}} > 0$ if $d_t^{\mathrm{obs}} < d_{\max}^{\mathrm{obs}}$, otherwise $C^{\mathrm{obs}} = 0$; $C^{\mathrm{col}} < 0$ if collision occurs, otherwise $C^{\mathrm{col}} = 0$; $C^{\mathrm{flag}} > 0$ if the valuable trajectory exists, otherwise $C^{\mathrm{flag}} = 0$.

As defined as (10) and (11), the main reward $f(t)$ is negatively correlated with the errors of the position and orientation between the robot end at the tth transition and that of the objective, which can motivate the robot to approach the objective. In addition, in (12), when the robot explores a feasible trajectory, the trajectory will be retained and considered as the valuable trajectory for adjusting the reward distribution, as shown in Fig. 1, the closer the subsequently explored node $\boldsymbol{w}_t$ is to the valuable trajectory, the lager its reward is. Through guiding by the adjusted reward, the valuable trajectory can be inherited and shared in subsequent exploration, which mitigates the impact of poor exploration and makes the policy learning efficient and robust.

$$\zeta = L + E \tag{13}$$

where ζ represents the disadvantage of the valuable trajectory, where both path length and energy consumption are considered with equal weight. The sum of their values is regarded as the disadvantage of the valuable trajectory. During later training, valuable trajectories are continuously updated according to (13). When the disadvantage ζ of the current valuable trajectory is larger than that of the newly explored one, the current valuable trajectory will be updated by the newly discovered one. This ensures that policy learning remains aligned with the most recent valuable trajectory. Details about TG are given in Algorithm 2.

Remark 4: Note that the valuable experience will not be retained after the tutoring (when $\rho_g = 0$). The subsequent trajectories explored by the agent will be unbiased, since the additional reward $g(t)$ adjusts the reward distribution according to the trajectory found independently by the agent, which allows the agent to seek the trajectory that is both length and energy optimal. Importantly, the valuable trajectory will be updated, which maintains that the valuable trajectory being shared is up-to-date. Moreover, the reward distribution can be updated along with the policy updating, which mitigates the impact of inappropriate rewards on the policy learning.

IV. Rapid Refitting Program

As is known, for DRL, even localized changes occur in the environment, it is generally necessary to retrain or replan. This greatly diminishes the generalization of the learned policy. To cope with such issue, a rapid refitting program is proposed, it contains a PP and an IKM, where the PP is based on Voronoi diagram [24], [25] and Dijkstra planner [26], and the IKM is a mixed normal distribution-based energy-optimal IK solver with an ANN.

As presented by (7), the information of each node in a trajectory contains positions, orientations, and angles. The path in the Cartesian space is described as the position set $\mathbf{P} = \{\boldsymbol{p}_1, \boldsymbol{p}_2, \ldots, \boldsymbol{p}_{\mathrm{M}}\}$, where $\boldsymbol{p}_m \in \mathbf{P}$ is the position at the

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 02,2026 at 09:12:06 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: TG mechanism.

Initialize the actor and critic networks, the tutoring memory, the long-term memory, $\boldsymbol{p}^{\mathrm{obj}}$, $\boldsymbol{o}^{\mathrm{obj}}$, τ, g, G_0^{ρ}, $\mathrm{G}_{\max}^{\rho} \in \{1, 2, \cdots, \mathrm{G}\}$, $\boldsymbol{a}_{\max}$, $d_{\max}^{\mathrm{obs}}$, $\mathrm{L}_{\max}$, $\mathrm{E}_{\max}$, C^{obs}, C^{col} and C^{flag};
for $g = 1, 2, \cdots, \mathrm{G}$ **do**
  Initialize state $\boldsymbol{s}_t$, where $t = 0$;
  for $t = 1, 2, \cdots, \mathrm{T}$ **do**
    Collect experience data including $\boldsymbol{a}_t$, $\boldsymbol{s}_{t+1}$ and reward r_{t+1} ;
    if successful trajectories are found and $\rho_g \neq 0$ **then**
      Simplify the trajectory by (5);
      Add the simplified data of the trajectory to the tutoring memory;
    Sample data from the tutoring and the long-term memories into the mini-batch according to ρ_g;
    Update the critic and actor networks with intervals by (1) and (2), respectively;
  if $\mathrm{G}_0^{\rho} \leq g \leq \mathrm{G}_{\max}^{\rho}/2$ **then**
    $\rho_g \leftarrow \rho_g + \tau$;
  else if $\mathrm{G}_{\max}^{\rho}/2 < g < \mathrm{G}_{\max}^{\rho}$ **then**
    $\rho_g \leftarrow \rho_{\max} - \tau$;
  if valuable trajectories are found and $\rho_g = 0$ **then**
    Adjust the reward distribution according to Eqs. (8)-(12).

mth node. The orientation set can be expressed as $\boldsymbol{\Upsilon} = \{\boldsymbol{\upsilon}_1, \boldsymbol{\upsilon}_2, \ldots, \boldsymbol{\upsilon}_{\mathrm{M}}\}$, where $\boldsymbol{\upsilon}_m \in \boldsymbol{\Upsilon}$ and $\boldsymbol{\upsilon}_m \in \mathbb{R}^{3\times3}$ is the orientation of the robot end at the mth node. The joint angle set is $\boldsymbol{\Theta} = \{\boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \ldots, \boldsymbol{\theta}_{\mathrm{M}}\}$, where $\boldsymbol{\theta}_m \in \boldsymbol{\Theta}$ is the joint angles at the mth node. If the nodes from $\boldsymbol{p}_{m+1}$ to $\boldsymbol{p}_{m+n-1}$ ($n \in \{2, \ldots, \mathrm{M} - m\}$) are invalid due to localized changes in the environment, PP and IKM will be activated.

A. PP for Path Refitting

This article aims to rapidly obtain a feasible path for locally changed environments in a simple way. The Dijkstra planner is a typical and low-complexity path planner widely used [27]. It needs sufficient candidate nodes that are obtained by randomly sampling or griding to obtain a high-quality path. However, it is cumbersome and time-consuming for the Dijkstra planner to explore the entire region, which inevitably generates needless explorations and leads to slow convergence.

Voronoi diagram features high-computational efficiency and is often used to assist traditional path planners in providing appropriately candidate nodes surrounding the obstacles [24], [25]. Dijkstra is employed to search for feasible paths based on these candidate nodes, and the iterative optimization process (as illustrated in *Step 4* below) is designed to progressively reduce path length.

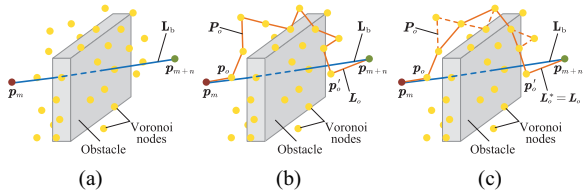


Fig. 3. Schematic diagram of the proposed PP. (a) Voronoi diagram is used to generate candidate nodes surrounding the obstacles. Nodes $\boldsymbol{p}_m$ and $\boldsymbol{p}_{m+n}$ on the previous path are defined as the starting and ending nodes, respectively, and are connected by a straight line labeled $\mathbf{L}_{\mathrm{b}}$. (b) $\boldsymbol{p}_o$ and $\boldsymbol{p}'_o$ are the candidate nodes closest to $\boldsymbol{p}_m$ and $\boldsymbol{p}_{m+n}$, respectively, and are defined as the temporary starting and ending nodes. The Dijkstra planner is utilized to plan a path from $\boldsymbol{p}_o$ and $\boldsymbol{p}'_o$ based on these candidate nodes, and this path is labeled $\boldsymbol{P}_o$. (c) $\boldsymbol{P}_o$ will be iteratively optimized according to (5) for shorter paths.

The main principles of PP can be depicted in Fig. 3, and the steps in detail are elaborated as follows.

Step 1: Assuming $\mathbf{P}$ is the previous path, and there are local changes occur in the environment resulting in some invalid position nodes with sequence number from $m+1$ to $m+n-1$. $\boldsymbol{p}_m$ and $\boldsymbol{p}_{m+n}$ belonging to $\mathbf{P}$ are considered as the starting node and the ending one, respectively. Connect them by a straight line as a basic path $\mathbf{L}_{\mathrm{b}}$ as shown in Fig. 3(a).

Step 2: Voronoi diagram is used to obtain candidate nodes surrounding the obstacle. On the two sides of the obstacle, where $\mathbf{L}_{\mathrm{b}}$ penetrates into and out of the obstacle, as shown in Fig. 3(b), select two nodes $\boldsymbol{p}_o$ and $\boldsymbol{p}'_o$ that have the smallest distance from $\boldsymbol{p}_m$ and $\boldsymbol{p}_{m+n}$, as the temporary starting and ending nodes, respectively, where $o \in \{1, 2, \ldots, \mathrm{O}\}$.

Step 3: In Fig. 3(b), the Dijkstra planner is used to plan the segment path $\boldsymbol{P}_o$ from $\boldsymbol{p}_o$ to $\boldsymbol{p}'_o$ based on the candidate nodes that can bypass the obstacle.

Step 4: Connect $\boldsymbol{P}_o$ with $\boldsymbol{p}_m$ and $\boldsymbol{p}_{m+n}$ at two sides for an entire path $\boldsymbol{L}_o$ to replace $\mathbf{L}_{\mathrm{b}}$. The total number of the nodes of $\boldsymbol{L}_o$ is Δ, $\boldsymbol{p}_m = \boldsymbol{L}_{o,1}$, and $\boldsymbol{p}_{m+n} = \boldsymbol{L}_{o,\Delta}$. From $\boldsymbol{L}_{o,1}$, try to directly connect the node afterward with an interval δ ($\delta \in \{2, \ldots, \Delta - 2\}$) until $\delta = \Delta - 2$ or failure. If failed, restart from $\boldsymbol{L}_{o,\delta-1}$, and keep trying to connect afterward notes. This step will be looped until $\boldsymbol{L}_o$ becomes unchanging, and as shown in Fig. 3(c), $\boldsymbol{L}_o^* = \boldsymbol{L}_o$ is the final path.

Step 5: Loop Steps 2–4 until all obstacles are bypassed.

The proposed PP only explores the regions surrounding the obstacles rather than the entire region, which allows it to save a considerable cost in exploring. Moreover, candidate nodes generated by the Voronoi diagram diminish the bias of randomness and significantly boost the efficiency of the path planning.

B. IKM for IK Solving

After $\boldsymbol{L}_o^*$ is obtained by the PP, the nodes in Cartesian space need to be mapped into the joint space. IKM, containing an ANN-based IK mapper and a mixed normal distribution-based algorithm, is proposed.

The pose transformation matrix for the mth node on the path can be expressed as

$$\boldsymbol{T}_m = \begin{bmatrix} \boldsymbol{\upsilon}_m & \boldsymbol{p}_m \\ \mathbf{0} & \mathbf{1} \end{bmatrix}. \tag{14}$$

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 02,2026 at 09:12:06 UTC from IEEE Xplore. Restrictions apply.

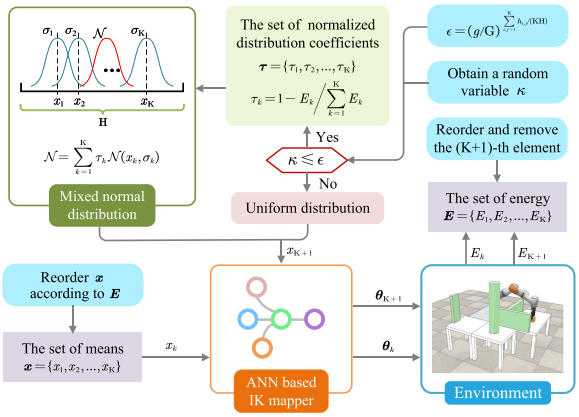


Fig. 4. Schematic diagram of the proposed IKM. The number of elements in the mean set $\boldsymbol{x}$ and energy consumption $\boldsymbol{E}$ is maintained at K. The κ is randomly selected and compared against the threshold ϵ. If $\kappa \leq \epsilon$, $x_{\mathrm{K}+1}$ follows the mixed normal distribution; otherwise, $x_{\mathrm{K}+1}$ follows the uniform distribution. The corresponding $E_{\mathrm{K}+1}$ is obtained, and its elements are sorted in ascending order. Simultaneously, the elements in $\boldsymbol{x}$ are sorted according to the corresponding elements in $\boldsymbol{E}$. At the end of the trial, E_1 represents the minimum energy consumption.

The forward kinematics can be defined as

$$f(\boldsymbol{\theta}_m) \cong \boldsymbol{T}_m. \tag{15}$$

Thus, the IK can be described as

$$\boldsymbol{\theta}_m = f^{-1}(\boldsymbol{T}_m). \tag{16}$$

The ANN-based IK mapper is used to approximate f^{-1} with the loss function defined as

$$L_f = \sqrt{\sum_{i=1}^{4}\sum_{j=1}^{4}\left[\hat{\boldsymbol{T}}_m(i,j) - \boldsymbol{T}_m(i,j)\right]^2} \tag{17}$$

where $\hat{\boldsymbol{T}}_m = f(\boldsymbol{\theta}_m)$ is the estimated pose matrix provided by the mapper. $\hat{\boldsymbol{T}}_m(i,j)$ denotes the element of row i and column j in $\hat{\boldsymbol{T}}_m$, and $\boldsymbol{T}_m(i,j)$ also has the same representation in $\boldsymbol{T}_m$.

The proposed IKM can be expressed as shown in Fig. 4 and Algorithm 3, where a positive correlation relationship is created between the epoch number g and ϵ. In the early stage of the iterations, ϵ is small, it is promised that $x_{\mathrm{K}+1}$ is more likely to follow the uniform distribution and the mapper prefers exploratory behaviors. Accordingly, in the late stage of the iterations, the output of the mapper is likely to be close enough to the optimal solution, then excessive exploration will be unnecessary because it will diminish the efficiency of seeking the energy-optimal solution. Consequently, it is preferred that $x_{\mathrm{K}+1}$ follows the mixed normal distribution $\mathcal{N}$ to enhance exploitation of the previous iteration results. In addition, in each epoch, $\boldsymbol{E}$ will be sorted, and in this way, the energy-optimal IK solution can be maintained and updated.

Remark 5: When localized changes occur, the environment could be simplified or complicated, in both cases PP and IKM will be activated. The proposed PP can rapidly find alternative paths, and the energy-optimal IK can be efficiently mapped by the proposed IKM. In a nutshell, without retraining DRL, using the proposed rapid refitting program composed of PP and IKM can assist the agent to rapidly adapt to newly changed environment, which significantly enhances the generalization performance of the learned policy.

Algorithm 3: IKM.

Input : $\mathbf{P}_{m,m+n}$

Initialize the ANN-based mapper, the set of the means $\boldsymbol{x} = \{x_1, x_2, \cdots, x_{\mathrm{K}}\}$, constants σ_k ($k \in \{1, 2, \cdots, \mathrm{K}\}$), G and H, where K is the total number of the normal distributions, σ_k is standard deviation of each normal distribution, G is the total number of the epochs and H is the maximum distance between two means;

for $g = 1, 2, \cdots, \mathrm{G}$ **do**

  $\boldsymbol{x}$ is used as the input of the mapper, and obtain $\boldsymbol{E} = \{E_1, E_2, \cdots, E_{\mathrm{K}}\}$ through (9) and the set of distribution coefficients $\boldsymbol{\tau} = \{\tau_1, \tau_2, \cdots, \tau_{\mathrm{K}}\}$ through the mapper, where $\tau_k = 1 - E_k / \sum_{k=1}^{\mathrm{K}} E_k$, then $\boldsymbol{\tau}$ is normalized;

  Calculate the mixed normal distribution $\mathcal{N} = \sum_{k=1}^{\mathrm{K}} \tau_k \mathcal{N}(x_k, \sigma_k)$;

  Calculate $h_{i,j}$ as the distance between x_i and x_j, where $i, j \in \{1, 2, \cdots, \mathrm{K}\}$ and $i \neq j$. Obtain a random variable κ and calculate a threshold ϵ by $\epsilon = (g/\mathrm{G})^{\sum_{i,j=1}^{\mathrm{K}} h_{i,j}/(\mathrm{KH})}$;

  if $\kappa \leq \epsilon$ **then**

    Sample $x_{\mathrm{K}+1}$ following the mixed normal distribution $\mathcal{N}$;

  else

    Sample $x_{\mathrm{K}+1}$ following the uniform distribution;

  Add $x_{\mathrm{K}+1}$ to the set $\boldsymbol{x}$. Obtain $E_{\mathrm{K}+1}$ by the mapper and add it to the set $\boldsymbol{E}$;

  Reorder $\boldsymbol{E}$;

  Remove $E_{\mathrm{K}+1}$ and the corresponding $x_{\mathrm{K}+1}$;

The nodes where IK solving fails are marked as blocked and are considered as obstacles;

Output: The lowest energy consumption $E_{\min}$ and the corresponding $\boldsymbol{\theta}_{\min}$.

From a systemic perspective, as shown in Fig. 5, the proposed DEAC is an actor-critic structure of the off-policy model-free DRL algorithm. The entropy is adjusted according to the TD error, with a smaller TD error indicating more adequate exploration. When v is less than H_v, the entropy is adjusted to encourage exploration, while H_v is reduced to impose a constraint on the return to ensure that the reward and the target are primary in the return. In the early training, successful trajectories are collected in the TG mechanism. *Tutoring* is used to generate heuristic trajectories based on these successful trajectories to facilitate exploration of potentially valuable trajectories, and *Guiding* is used to adjust the reward distribution so as to reduce the forgetting of those valuable trajectories. Moreover, in case localized changes occur in the environment, the proposed rapid refitting program will be activated to handle the effect of this change on the planning and update the whole trajectory. In the

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 02,2026 at 09:12:06 UTC from IEEE Xplore. Restrictions apply.

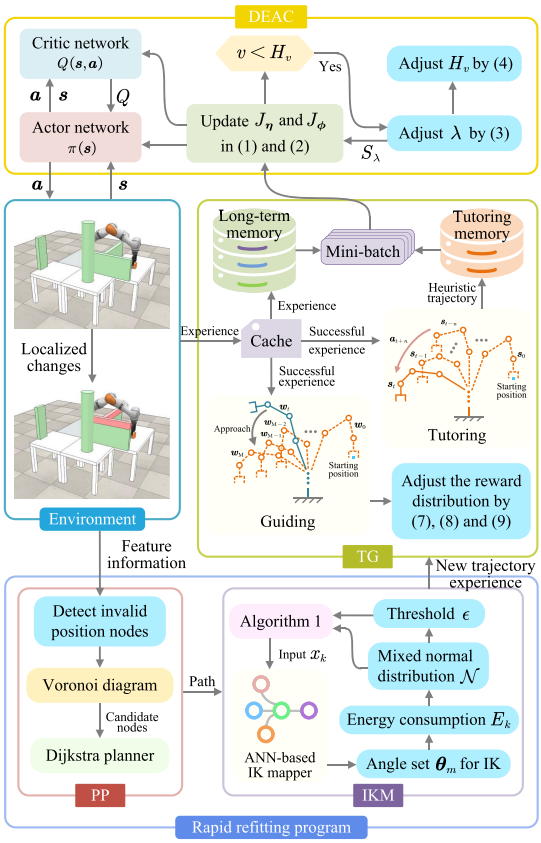


Fig. 5. Schematic diagram of the proposed motion planning framework. DEAC features efficient policy learning by adjusting entropy. TG is used to assist DEAC in exploring and exploiting potential valuable experience by generating heuristic trajectory data and adjusting the reward distribution. The rapid refitting program incorporating PP and IKM is used to cope with changes occurring in the environment and provide the robot with alternative trajectories that does not require retraining.

rapid refitting program, PP, which is a combination of Voronoi diagram and Dijkstra planner, generates paths in Cartesian space, then IKM is used to map these paths to trajectories with optimal energy consumption in the joint space. The proposed methods can be integrated into a rapid generalization framework for robot motion planning to cope with locally changed environments.

V. Experimental Results

In this article, six OpenAI MuJoCo robotic benchmarks are selected and six customized robotic scenarios are established to verify the proposed methods, and the experimental results are presented and discussed. All the algorithms involved are developed and implemented on the Ubuntu 18.04 LTS operating system, and run on a laptop platform equipped with an Intel Core i7-8750H CPU (at 2.20 GHz), 8-GB DDR4 RAM, and an NVIDIA GeForce GTX 1050Ti with 4-GB GDDR5 VRAM.

A. Verification of the Proposed DEAC Algorithm

The proposed DEAC is compared with two recent DRL algorithms (soft actor-critic (SAC) [9] and TAC-CA [10]) in the standard robotics tasks of OpenAI MuJoCo to certify the superiority.

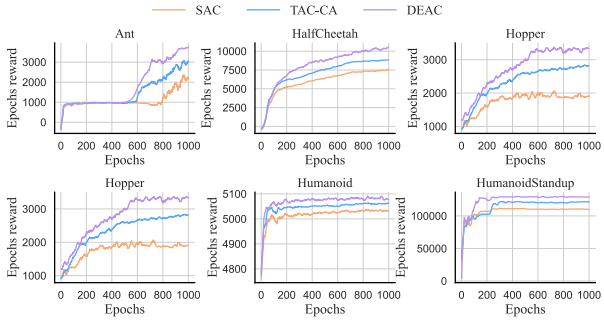


Fig. 6. Comparisons of DRL algorithms in the OpenAI MuJoCo standard robotics tasks. The capability of each algorithm to obtain episode reward is compared, and the curves are smoothed with convolution.

TABLE I
Episode Reward Comparison of Different DRL Algorithms in the OpenAI MuJoCo Standard Robotics Tasks

Episodes	1∼500			501∼1000		
	SAC	TAC-CA	DEAC	SAC	TAC-CA	DEAC
Ant	915.23	**921.56**	914.42	1276.58	2010.13	**2668.90**
HalChe	4869.90	5714.31	**6495.58**	7115.71	8430.27	**9755.57**
Hopper	1624.69	2009.93	2268.51	1914.02	2713.39	**3273.81**
Pusher	−373.72	−387.42	**−308.88**	−197.26	−198.62	**−194.10**
Humanoid	5003.15	5035.51	**5059.79**	5030.52	5057.53	**5080.87**
HumSta	1043.90	1085.70	**1195.07**	1103.52	1208.45	**1294.45**

OpenAI Gym MuJoCo is a standardized platform for robotic tasks, some benchmarks [Ant, HalfCheetah (HalChe in short)], Hopper, Pusher, Humanoid, and HumanoidStandup (HumSta in short, and the episode reward is scaled by 0.01)] in it are selected as the experimental scenarios to verify the generality and efficiency of DEAC. For each benchmark, every DRL algorithm (SAC, TAC-CA, and DEAC) runs 1000 episodes and each episode has 1000 steps. As shown in Fig. 6 and Table I, the proposed DEAC performs superior to the other algorithms in terms of the episode reward. In detail, the trial of each algorithm in each scenario is split into the early (episode from 1 to 500) and the later (episode from 501 to 1000) stages. In the early stage, for each scenario, the rates of the episode reward earned by DEAC to those earned by SAC are 100.89% , 125.01% , 126.87% , 99.01% , 100.92% , and 116.97% , respectively, and the rates of the episode reward earned by DEAC to those earned by TAC-CA are 100.2% , 110.23% , 107.7% , 102.55% , 100.28% , and 113.12% , respectively. As shown in Fig. 6(a), (b), and (d), the exploratory ability of DEAC can be enhanced by (3), which enables the agent to obtain more significantly increasing episode reward than that obtained by the other algorithms. Moreover, in the later stage, for each scenario, the rates of the episode reward earned by DEAC to those earned by SAC are 155.38% , 128.56% , 147.16% , 100.08% , 100.88% , and 115.79% , respectively, and the rates of the episode reward earned by DEAC to those earned by TAC-CA are 118.89% , 112.97% , 117.7% , 100.77% , 100.34% , and 107.11% , respectively. To constrain excessively exploratory behaviors, as shown in Fig. 6(c) and (f), the TD error threshold is shrunk according to (4) to make the reward dominant in the episode return, thus the agent is motivated to obtain the maximum episode reward and accomplish the task, as well as the learned policy can be maintained and sufficiently exploited.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 02,2026 at 09:12:06 UTC from IEEE Xplore. Restrictions apply.

TABLE II
MAIN HYPERPARAMETERS IN THE EXPERIMENTS

	Main hyperparameters	Settings
DEAC	Learning rate	0.0001
	Discount factor	0.99
	Memory size	100000
	Mini-batch size	32
	Hidden units of networks	200^{20}*10
	Activation functions	ReLU6*ReLU6*ReLU
	Optimizer	AdamW
	Entropy	Tsallis
	The interval of λ	[0.1. 1.5]
	The interval of H_v	(0, 5]

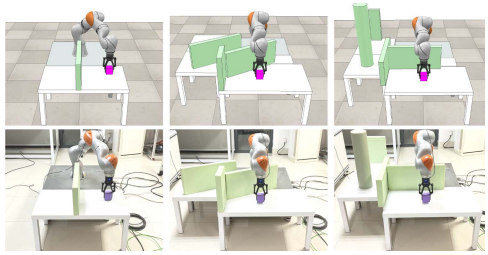


Fig. 7. Scenarios in simulation and the real-world platforms. The robotic task is to pick up the block from the starting place, and then, put it in the designated place. No collision with obstacles is allowed throughout. (a) Scenario 1. (b) Scenario 2. (c) Scenario 3.

The main hyperparameters of DEAC in these experiments are given in Table II.

B. Verification of the Proposed TG Mechanism

The proposed DEAC and TG for motion planning are experimentally verified on a 7-DoF redundant robot manipulator KUKA LBR 7 R800 iiwa, in different work scenarios, as shown in Fig. 7. The state and action are defined as

$$\boldsymbol{s} = \{\boldsymbol{\theta}, e^{\text{pos}}, e^{\text{ori}}\},\ \boldsymbol{\theta} = \{\theta_1, \theta_2, \ldots, \theta_{\text{D}}\} \tag{18}$$

$$\boldsymbol{a} = \Delta\boldsymbol{\theta} = \{\Delta\theta_1, \Delta\theta_2, \ldots, \Delta\theta_{\text{D}}\} \tag{19}$$

where $\boldsymbol{\theta}$ denotes the set of the joint angles of the robot and $\Delta\boldsymbol{\theta}$ is the increments for the joint angles. D is the number of the joints. An attenuated noise is added to $\boldsymbol{a}$ in the experiments.

TG is integrated into DEAC as DEAC-TG, and the proposed DEAC and DEAC-TG are compared with two state-of-the-art methods (DRL with base controllers (DRLwB) [28] and back-stepping experience replay (BER) [29]). For fairness, the DRL in all the methods is replaced with DEAC. Every method runs 2000 episodes, 300 steps in each episode, for each scenario.

As shown in Fig. 8, the proposed DEAC-TG performs the best in episode reward as shown in Fig. 8(a), (d) and (g), due to the superiority of DEAC in balancing exploration and exploitation in the episode return, which enables the agent to efficiently exploit the enhanced exploratory ability to obtain the maximum episode reward. Moreover, benefiting from the TG mechanism, the tutoring can generate heuristic trajectory by using explored experience to reduce the path length, as shown in Fig. 8(b), (e), and (h); the proposed DEAC-TG performs superior to the other methods. In addition, after a feasible policy has been learned, the guiding of TG will be used to prevent the previous valuable experience from being neglected by adjusting the reward distribution. In the meantime, as described in (12), the guiding motivates the agent to explore trajectories that are more energy-saving, as illustrated in Fig. 8(c), (f), and (i) that the proposed DEAC-TG produces the energy-optimal trajectory among all the methods.

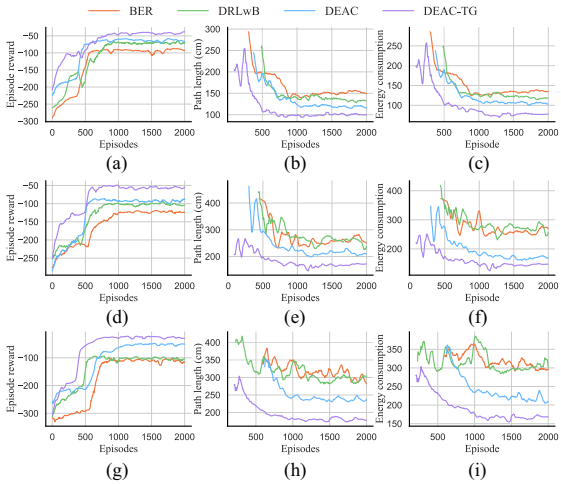


Fig. 8. Performance comparisons in terms of episode reward, path length, and energy consumption in different Scenarios: (a)–(c) for Scenario 1, (d)–(f) for Scenario 2, and (g)–(i) for Scenario 3. The curves are smoothed with convolution.

TABLE III
PLANNING RESULTS IN DIFFERENT SCENARIOS

		BER	DRLwB	DEAC	DEAC-TG
Scenario 1	$L_{\min}$(cm)	128.77	114.58	104.47	**79.15**
	$E_{\min}$	116.57	104.71	89.19	**63.10**
	$\xi(\%)$	87.3	**95.8**	90.8	90.7
Scenario 2	$L_{\min}$(cm)	218.74	198.54	191.00	**151.72**
	$E_{\min}$	226.75	188.11	153.78	**132.80**
	$\xi(\%)$	80.1	83.4	87.6	**91.6**
Scenario 3	$L_{\min}$(cm)	265.94	263.07	220.30	**162.37**
	$E_{\min}$	267.44	262.23	195.98	**151.95**
	$\xi(\%)$	78.6	82.7	89.3	**90.5**

The purple bold values are the best performances (maximum or minimum values).

The best policy of each method is selected for the testing, where there are 1000 episodes and each episode has 300 steps. The comparative results are presented in Table III, where $L_{\min}$ represents the minimum path length defined by $L_{\min} = \min\{L_1, L_2, \ldots, L_{\text{G}}\}$ and elements in it are calculated by (8), the $E_{\min}$ denotes the minimum energy consumption defined by $E_{\min} = \min\{E_1, E_2, \ldots, E_{\text{G}}\}$ and elements in it are calculated by (9) and ξ is the success rate defined by $\xi = \frac{1}{\text{G}}\sum_{g=1}^{\text{G}} \mu_g \times 100\%$ as the percentage of successful times of reaching the objective in total episodes where $\mu_g = 1$ if the agent reaches the objective, otherwise $\mu_g = 0$.

The DRLwB in [28] uses expert policies to assist the agent in the policy learning, where the learning efficiency is heavily dependent on the quality of the expert policies, and the policy learned is likely to be biased. The BER in [29] aims to accelerate convergence in the early training through using bidirectional transitions, the back-stepping transition function

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 02,2026 at 09:12:06 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
MAIN HYPERPARAMETERS IN THE EXPERIMENTS

	Main hyper-parameters	Settings
TG	Capacity of the tutoring memory	50000
	Capacity of the long-term memory	100000
	Maximum action angle $\boldsymbol{a}_{\max}$	3
	Maximum distance under consideration $d_{\max}^{\text{obs}}$	0.15
	Collision penalty C^{col}	−0.5

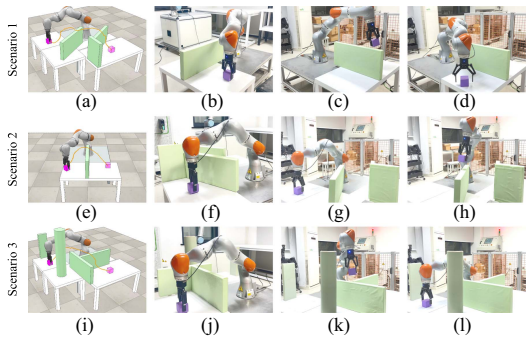


Fig. 9. Performance of the proposed DEAC-TG: (a) in simulation and (b)–(d) in real-word platform for Scenario 1; (e) in simulation and (f)–(h) in real-word platform for Scenario 2; (i) in simulation and (j)–(l) in real-word platform for Scenario 3.

TABLE V
COMPARISONS OF DIFFERENT PLANNERS IN SCENARIOS 1–6

		1	2	3	4	5	6
HIA	$L_{\min}(\text{cm})$	95.97	178.05	165.32	142.93	187.07	202.28
	$\xi(\%)$	92.3	95.6	**98.6**	87.9	88.6	91.2
MIT*	$L_{\min}(\text{cm})$	85.39	160.24	163.46	133.58	170.49	185.13
	$\xi(\%)$	95.5	95.1	93.5	**89.6**	87.0	88.2
PP	$L_{\min}(\text{cm})$	**84.07**	**155.34**	**157.47**	**130.08**	**166.67**	**179.83**
	$\xi(\%)$	**96.3**	**97.6**	97.9	88.3	**90.1**	**91.5**

The purple bold values are the best performances (maximum or minimum values).

is computationally expensive or even unsolvable. In contrast, the proposed DEAC-TG can exploit the explored experience to generate heuristic data without extra data or expert policies, and the previous valuable experience can be shared in subsequent explorations. Moreover, the tutoring can be inherited by the guiding, which allows the path length and the energy consumption to be optimized simultaneously in the policy learning according to (10)–(12), as shown in Table III; the proposed DEAC-TG has the superiority to obtain a shorter path, less energy consumption, and a higher success rate. The main hyperparameters of DEAC are the same as that in the Table II, and the main hyperparameters of TG in these experiments are given in Table IV.

As presented in Fig. 9, the proposed motion planning approach DEAC-TG is experimentally verified in Scenario 1, Scenario 2, and Scenario 3 in the real-word platforms.

C. Verification of the Proposed PP and IKM

The proposed PP is compared with current typical path planners (hybrid improved algorithm (HIA) in [30] and multiinformed trees (MIT$*$) in [31]) for path planning tasks in Scenarios 1–6. As shown in Table V, MIT$*$ employs an adaptive sampler that dynamically adjusts the sampling strategy based on the exploration process, but its path optimization relies on randomly sampled nodes, requiring a large number of iterations to approximate the theoretical shortest path. HIA is an integrated approach that can generate high-quality paths under specific conditions. Nonetheless, potential parameter conflicts among its submodules pose a challenge to its generalization. PP can find shorter path lengths than the other two planners in Scenarios 1–6, as the progressive mechanism can remove unnecessary nodes to reduce path length. Furthermore, since Voronoi diagrams can supply feasible candidate nodes for Dijkstra, PP also achieves higher success rates.

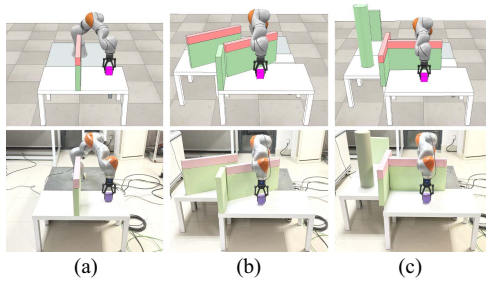


Fig. 10. Scenarios in simulation and the real-word platforms. Scenario 4 is locally changed from Scenario 1, Scenario 5 is locally changed from Scenario 2, and Scenario 6 is locally changed from Scenario 3. In each scenario, the robotic task is to pick up the block from the starting place, and then, put them in the designated place. No collision with obstacles is allowed throughout. (a) Scenario 4. (b) Scenario 5. (c) Scenario 6.

TABLE VI
COMPARISONS OF DIFFERENT IK SOLVERS IN SCENARIOS 1–6

		1	2	3	4	5	6
PCmI	$E_{\min}$	128.21	226.28	301.33	212.35	236.69	318.75
	$\xi(\%)$	96.7	90.2	85.5	96.1	**89.4**	86.3
TFOA	$E_{\min}$	112.65	201.29	198.68	189.69	177.65	223.58
	$\xi(\%)$	97.3	**92.1**	**86.1**	93.3	87.2	87.0
IKM	$E_{\min}$	**69.47**	**143.10**	**172.36**	**123.30**	**143.99**	**167.15**
	$\xi(\%)$	**98.1**	89.6	85.2	**97.5**	88.5	**87.2**

The purple bold values are the best performances (maximum or minimum values).

As shown in Fig. 10, localized changes occur in Scenario 1 to form Scenario 4, in Scenario 2 to form Scenario 5, and in Scenario 3 to form Scenario 6.

As shown in Table VI, the proposed IKM is compared with the state-of-the-art IK solvers (a hybrid method based on pose errors, Cartesian movement primitives, a modified neural network and an image-guided navigation (PCmI) in [32] and the combination of a task-priority framework and an optimization-based approach (TFOA) in [33]) in these six scenarios, and all methods are based on identical paths provided by DEAC-TG. PCmI can rapidly acquire expert skills through imitation learning, but it could be constrained by the experience of the expert and unable to autonomously explore IK with lower energy consumption. Although TFOA has a fast response, it requires disabling optimization tasks to adjust the nullspace, making it difficult to ensure low energy consumption while tracking trajectories. In the proposed IKM, the mapper has strong nonlinear fitting capability, enabling IKM to be theoretically applied to arbitrary manipulator and improve success rates. Furthermore,

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 02,2026 at 09:12:06 UTC from IEEE Xplore. Restrictions apply.

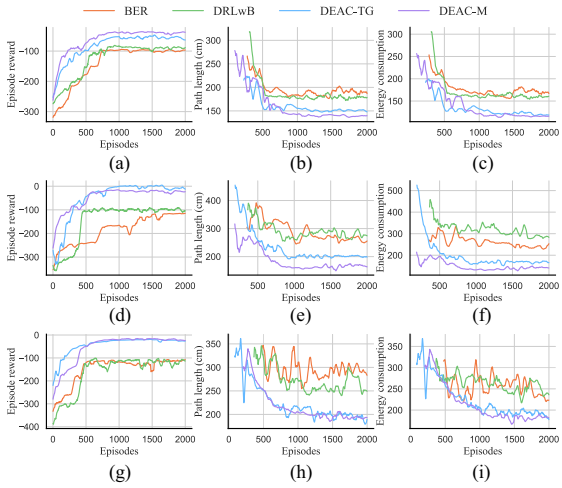


Fig. 11. Performance comparisons in terms of episode reward, path length, and energy consumption in different Scenarios: (a)–(c) for Scenario 4, (d)–(f) for Scenario 5, and (g)–(i) for Scenario 6. The curves are smoothed with convolution. DEAC-M inherits the policy of DEAC-TG in each episode without retraining.

TABLE VII
PLANNING RESULTS IN LOCALLY CHANGED SCENARIOS

		BER	DRLwB	DEAC-TG	DEAC-M
Scenario 4	$L_{\min}$(cm)	161.75	150.97	141.88	**130.67**
	$E_{\min}$	146.70	136.92	**102.06**	106.99
	ξ(%)	83.8	**87.3**	86.8	87.2
Scenario 5	$L_{\min}$(cm)	241.77	239.11	163.53	**159.21**
	$E_{\min}$	212.87	264.73	137.58	**123.76**
	ξ(%)	70.5	73.4	**85.6**	84.3
Scenario 6	$L_{\min}$(cm)	232.64	224.44	156.62	**152.69**
	$E_{\min}$	201.64	192.85	157.96	**151.06**
	ξ(%)	74.9	75.6	85.5	**86.7**

The purple bold values are the best performances (maximum or minimum values).

the mixed normal distribution-based energy optimization algorithm enables IKM to perform superior in minimizing energy consumption.

Integrate the proposed PP and IKM mechanisms into the DEAC-TG to establish a general framework, named as DEAC-M, and compare it with the state-of-the-art motion planning methods DRLwB in [28] and BER in [29]. Considering that the proposed DEAC-M does not need to retrain the DRL in locally change environments, the corresponding policy in each episode to be applied in Scenarios 4–6 will inherit the DEAC-TG's policy learned in Scenarios 1–3, respectively.

As shown in Fig. 11 and Table VII, the proposed DEAC-TG performs better than DRLwB and BER in Scenarios 4–6. In terms of the episode reward, as shown in Fig. 11(a), (d), and (g), the proposed DEAC-M does not have an advantage over the proposed DEAC-TG, since PP and IKM aims to refit the trajectory and the episode reward is not taken into account. As shown in Fig. 11(b), (e), and (h) and Table VII, DEAC-M performs shorter paths than DEAC-TG, which is attributed to the Voronoi diagram and Dijkstra planner-based PP that can minimize the path length while bypassing the obstacles. Moreover, since IKM is capable of seeking energy-optimal IK solutions, as shown in Fig. 11(c), (f), and (i) and Table VII, DEAC-M has the edge in terms of energy consumption in the comparison with the other methods. In addition, the proposed DEAC-M and DEAC-TG have maintained a high level of success rates since PP has the ability of the systematic exploration and IKM features strong nonlinear fitting.

TABLE VIII
RUNTIME COMPARISONS IN DIFFERENT SCENARIOS

(Unit: minutes)	BER	DRLwB	DEAC	DEAC-TG	DEAC-M
Scenario 1	**267.6**	319.2	279.3	292.5	–
Scenario 2	**270.8**	316.9	275.2	296.4	–
Scenario 3	271.2	323.7	**271.0**	290.3	–
Scenario 4	261.3	311.6	–	287.1	**25.7**
Scenario 5	274.6	320.1	–	294.6	**36.7**
Scenario 6	269.2	318.7	–	291.5	**36.2**

The purple bold values are the best performances (maximum or minimum values).

TABLE IX
MAIN HYPERPARAMETERS IN THE EXPERIMENTS

	Main hyper-parameters	Settings
PP and IKM	Maximum number of attempts	5000
	Hidden units of networks	100*100
	Activation functions	Tanh*Tanh*Tanh
	Learning rate	0.0001
	Optimizer	Adam
	IK solving threshold	10^{-4}

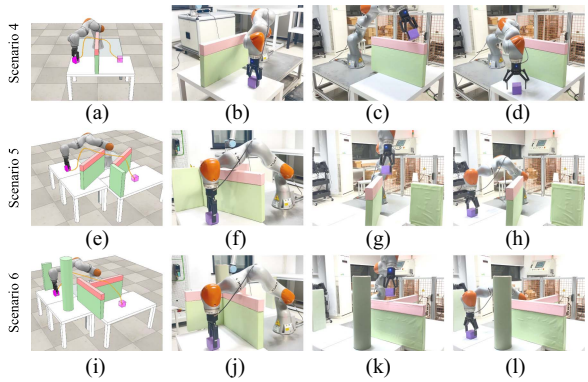


Fig. 12. Performance of the proposed DEAC-M: (a) in simulation and (b)–(d) in real-word platform for Scenario 4; (e) in simulation and (f)–(h) in real-word platform for Scenario 5; and (i) in simulation and (j)–(l) in real-word platform for Scenario 6.

As shown in Table VIII, since DRLwB has expert policies, DEAC is equipped with SAM, and DEAC-TG is assisted by both SAM and TG mechanism, they consume more training time compared to BER. However, when local changes occur in the environment, only DEAC-M does not require retraining, as its PP and IKM can rapidly generate alternative trajectories to enhance the efficiency of the agent in accomplishing tasks.

The main hyperparameters of PP and IKM in these experiments are given in Table IX. As shown in Fig. 12, the proposed motion planning framework DEAC-M is experimentally verified on the real-world platforms.

VI. CONCLUSION

This article discussed the problem of motion planning in the environments with localized changes. We showed that the proposed DEAC can automatically adjust the coefficient

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 02,2026 at 09:12:06 UTC from IEEE Xplore. Restrictions apply.

of the entropy cluster to boost the efficiency of the policy learning. A TG mechanism was proposed to generate heuristic experience without extra data or expert policies and adjust the reward distribution for the later updates according to the previous valuable experience. Moreover, TG can be used in an arbitrary DRL algorithm to enhance its performance in motion planning tasks. Furthermore, to cope with localized changes in the environment, PP and IKM were designed for locally refitting the trajectory with the length-optimal path and the energy-optimal IK solution. The proposed techniques can be combined as a framework called DEAC-M that generalizes motion planning in the environments with localized changes for different robots and/or different tasks using DRL.

The proposed motion planning scheme was experimentally verified on a 7-DoF serial robot manipulator. The experimental results showed that, as an important part of the proposed general framework DEAC-M, the proposed DEAC can motivate the agent to efficiently obtain high episode reward by balancing exploration and exploitation. Besides, the proposed TG mechanism can facilitate the agent to seek the trajectory that is both length and energy optimal by sufficiently utilizing the previous valuable experience. In addition, the proposed PP and IKM can be activated when the localized changes occur in the environment, which is a shortcut without retraining DRL to refit the trajectory while having similar edges with the retrained one in terms of path length, energy consumption and success rate. One more thing, the proposed methods can be generalized to robots with arbitrary geometries. Furthermore, to meet the diverse demands of future robot applications, the future work will be extended to collaborative tasks involving multitype robots, especially the human–robot collaboration scenarios. Embodied intelligence will also be introduced to enable robots to perceive physical environments and understand human intentions in real time during interaction, making human–robot collaboration more natural.

REFERENCES

[1] Q. Zhou, Y. Niu, W. Xiang, and L. Zhao, "A novel reinforcement learning algorithm based on broad learning system for fast communication antijamming," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2590–2599, Mar. 2025.

[2] W. Zhang, Y. Zhang, N. Liu, K. Ren, and G. Peng, "Dimension-variable mapless navigation with deep reinforcement learning," *IEEE/ASME Trans. Mechatron.*, to be published, doi: 10.1109/TMECH.2025.3541797.

[3] H. Liu, X. Li, M. Dong, Y. Gu, and B. Shen, "Robotic motion planning based on deep reinforcement learning and artificial neural networks," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 8465–8479, 2025.

[4] H. Hua, Y. Wang, H. Zhong, and Y. Fang, "Deep reinforcement learning-based hierarchical motion planning strategy for multirotors," *IEEE Trans. Ind. Inform.*, vol. 21, no. 6, pp. 4324–4333, Jun. 2025.

[5] P. Chen et al., "An environmental-complexity-based navigation method based on hierarchical deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2024, pp. 5119–5125.

[6] X. Liu, Z. Liu, G. Wang, Z. Liu, and P. Huang, "Efficient reinforcement learning method for multi-phase robot manipulation skill acquisition via human knowledge, model-based, and model-free methods," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 6643–6652, 2025.

[7] X. Xing, Z. Zhou, Y. Li, B. Xiao, and Y. Xun, "Multi-UAV adaptive cooperative formation trajectory planning based on an improved algorithm of deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 73, no. 9, pp. 12484–12499, Sep. 2024.

[8] D. Liu, Y. Jiang, Y. Du, Z. Wang, and M. Cong, "Bi-phase episodic memory-guided deep reinforcement learning for robot skills," *IEEE Robot. Autom. Lett.*, vol. 10, no. 9, pp. 8674–8681, Sep. 2025.

[9] T. Haarnoja et al., "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.

[10] X. Li, H. Liu, and M. Dong, "A general framework of motion planning for redundant robot manipulator based on deep reinforcement learning," *IEEE Trans Ind. Inform.*, vol. 18, no. 8, pp. 5253–5263, Aug. 2022.

[11] X. Hai, L. Tan, Q. Feng, H. Duan, and C. Wen, "Replanning-oriented framework for efficient real-time decision-making in multi-systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 7, pp. 5127–5137, Jul. 2025.

[12] W. Liang, Y. Liu, J. Wang, and Z.-X. Yang, "Trajectory progress-based prioritizing and intrinsic reward mechanism for robust training of robotic manipulations," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 9829–9842, 2025, doi: 10.1109/TASE.2024.3513354.

[13] T. Haarnoja et al., "Learning to walk via deep reinforcement learning," *Robot., Sci. Syst.*, pp. 1–10, 2019.

[14] Z. He, P. Chen, H. Shi, J. Li, and K.-S. Hwang, "Robotic locomotion skill learning using unsupervised reinforcement learning with controllable latent space partition," *IEEE Trans. Ind. Inform.*, vol. 21, no. 1, pp. 902–911, Jan. 2025.

[15] Y. Zang et al., "Robotic disassembly skill acquisition based on reinforcement learning with external knowledge injection," *IEEE Trans. Ind. Inform.*, vol. 21, no. 6, pp. 4576–4585, Jun. 2025.

[16] J. Chen, Y. Wu, and H. Qiao, "Memory, attention, and muscle synergies based reinforcement and transfer learning for musculoskeletal robots under imperfect observation," *IEEE/ASME Trans. Mechatron.*, vol. 30, no. 3, pp. 1853–1864, Jun. 2025.

[17] L. Jin, Y. Men, R. Song, F. Li, Y. Li, and X. Tian, "Robot skill generalization: Feature-selected adaptation transfer for peg-in-hole assembly," *IEEE Trans. Ind. Electron.*, vol. 71, no. 3, pp. 2748–2757, Mar. 2024.

[18] Y. Men, L. Jin, T. Cui, Y. Bai, F. Li, and R. Song, "Policy fusion transfer: The knowledge transfer for different robot peg-in-hole insertion assemblies," *IEEE Trans. Instrum. Meas.*, vol. 72, 2023, Art. no. 3528510.

[19] Z. Zhu, K. Lin, A. K. Jain, and J. Zhou, "Transfer learning in deep reinforcement learning: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 11, pp. 13344–13362, Nov. 2023.

[20] Y. Xu et al., "Multi-attention spatio-temporal fusion graph neural network for chemical process fault diagnosis," *IEEE Trans. Instrum. Meas.*, vol. 74, 2025, Art. no. 3530113.

[21] Y. Liu, R. Ni, and A. H. Qureshi, "Physics-informed neural mapping and motion planning in unknown environments," *IEEE Trans. Robot.*, vol. 41, pp. 2200–2212, 2025.

[22] M. A. Manzour, C. M. Elias, E. I. Morgan, and O. M. Shehata, "Development of a modular ROS-enabled pedestrian intention prediction architecture for maneuvering control," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 1, pp. 798–809, Jan. 2025.

[23] P. Foret et al., "Sharpness-aware minimization for efficiently improving generalization," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–19.

[24] Z. Wu, Y. Chen, J. Liang, B. He, and Y. Wang, "ST-FMT∗: A fast optimal global motion planning for mobile robot," *IEEE Trans. Ind. Electron.*, vol. 69, no. 4, pp. 3854–3864, Apr. 2022.

[25] J. Hu, Y. Hu, C. Lu, J. Gong, and H. Chen, "Integrated path planning for unmanned differential steering vehicles in off-road environment with terrains and obstacles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 6, pp. 5562–5572, Jun. 2022.

[26] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numer. Math.*, vol. 1, no. 1, pp. 269–271, 1959.

[27] Z. Qiu, I. Jokić, S. Tang, R. Noldus, and P. Van Mieghem, "Inverse all shortest path problem," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 2703–2714, May/Jun. 2024.

[28] G. Wang, M. Xin, W. Wu, Z. Liu, and H. Wang, "Learning of long-horizon sparse-reward robotic manipulator tasks with base controllers," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 3, pp. 4072–4081, Mar. 2024.

[29] X. Qi, D. Chen, Z. Li, and X. Tan, "Back-stepping experience replay with application to model-free reinforcement learning for a soft snake robot," *IEEE Robot. Autom. Lett.*, vol. 9, no. 9, pp. 7517–7524, Sep. 2024.

[30] Y. Su, J. Xin, and C. Sun, "Dynamic path planning for mobile robots based on improved RRT and DWA algorithms," *IEEE Trans. Ind. Inform.*, vol. 72, no. 10, pp. 10595–10604, Oct. 2025.

[31] L. Zhang et al., "Estimated informed anytime search for sampling-based planning via adaptive sampler," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 18580–18593, 2025.

[32] X. Jian et al., "Motion planning and control of active robot in orthopedic surgery by CAMP-based imitation learning and constrained optimization," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 12197–12212, 2025.

[33] H. Gamper, L. Rodrigo Pérez, A. Mueller, A. D. Rosales, and M. Di Castro, "An inverse kinematics algorithm with smooth task switching for redundant robots," *IEEE Robot. Autom. Lett.*, vol. 9, no. 5, pp. 4527–4534, May 2024.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 02,2026 at 09:12:06 UTC from IEEE Xplore. Restrictions apply.